

Assembly Line Monitoring System

A Complete Beginner's Guide

Computer Vision-Based Tower Assembly Process Monitoring

Table of Contents

1. Introduction and System Overview
2. Key Technologies Used
3. System Architecture
4. Imported Libraries Explained
5. Global Variables and Configuration
6. Core Functions Deep Dive
7. Assembly Process Steps
8. Web Server Implementation
9. Video Processing Pipeline
10. Detection Logic Explained
11. Troubleshooting and Best Practices

1. Introduction and System Overview

1.1 What Does This System Do?

This is an AI-powered computer vision system designed to monitor and track the assembly process of mechanical towers (likely for vehicles like trucks or buses). The system watches a live video feed of workers assembling parts and automatically verifies that each step is performed in the correct order.

1.2 Key Features

- Real-time video monitoring using RTSP camera streams
- AI-based object detection to identify parts and tools
- Sequence validation to ensure correct assembly order
- Web-based interface for remote monitoring
- Automatic recording capabilities
- Visual feedback with color-coded status indicators

1.3 How It Works (Simple Explanation)

Imagine having a smart assistant watching workers assemble parts. This assistant knows exactly what parts should be installed and in what order. When it sees a part being placed, it checks:

1. Is this the right part?
2. Is it being installed at the right time?
3. Has the previous step been completed?

If everything is correct, it gives a green checkmark. If something is wrong (like skipping a step), it shows a red warning.

2. Key Technologies Used

2.1 YOLO (You Only Look Once)

YOLO is an AI model that can identify objects in images or video frames. Think of it as teaching a computer to recognize things the way humans do, but much faster. In this system, YOLO has been trained to recognize specific parts like end covers, bolts, clamps, hammers, drill bits, and even hand positions.

2.2 Flask Web Framework

Flask is a Python tool that allows the system to run as a web server. This means you can view the monitoring system from any web browser on your network - no need to be physically at the monitoring station.

2.3 OpenCV (Computer Vision Library)

OpenCV is a library that helps process video streams - capturing frames, drawing rectangles around detected objects, adding text overlays, and converting video formats. It's like Photoshop, but for video processing and automated by code.

2.4 RTSP Video Streaming

RTSP (Real-Time Streaming Protocol) is how the system receives live video from network cameras. It's similar to watching a live YouTube stream, but designed for security and industrial cameras.

3. System Architecture

The system consists of several interconnected components working together:

Component	Function	Technology
Camera System	Captures live assembly video	RTSP Network Camera
Video Capture	Receives and processes video stream	PyAV (faster than OpenCV)
AI Detection	Identifies parts and tools	YOLO v8 Neural Network
Logic Engine	Validates assembly sequence	Custom Python Logic
Web Server	Provides remote access	Flask Framework
Display Interface	Shows live feed with overlays	HTML5 Video Streaming

3.1 Data Flow

1. Camera captures live video of assembly area
2. Video stream is received via RTSP protocol
3. Each frame is processed by YOLO AI models
4. Detected objects are analyzed by sequence logic
5. Visual overlays are added to show status
6. Processed video is streamed to web browsers
7. Optional: Video can be recorded for quality review

4. Imported Libraries Explained

Let's understand each library imported at the beginning of the code and why it's needed:

time

Handles time-related operations like measuring how long steps take and adding delays.

sys

Provides system-specific parameters and functions. Used here for path manipulation.

uuid

Generates unique identifiers (UUIDs). Used for creating unique video filenames.

queue

Implements thread-safe queues for managing video frames between threads.

threading

Allows running multiple operations simultaneously (multithreading).

cv2 (OpenCV)

The main computer vision library for video processing, image manipulation, and drawing.

skimage.metrics (SSIM)

Measures structural similarity between images (not actively used in current code).

av (PyAV)

Fast video processing library, used instead of OpenCV for better RTSP performance.

math

Provides mathematical functions, used here for calculating distances.

collections.deque

A double-ended queue, used for maintaining recent detection history.

Flask

Web framework that creates the HTTP server for browser access.

numpy

Numerical computing library, works with arrays and matrices (image data).

ultralytics (YOLO)

The AI object detection framework that powers the vision system.

5. Global Variables and Configuration

5.1 Assembly Steps Definition

The system tracks 16 specific assembly steps in order:

```
all_steps = [ "End cover fitment", # Step 0 "End cover bolt fitment", # Step 1 "Clamping", #  
Step 2 "Interlock pin fitment", # Step 3 "Hammering Interlock", # Step 4 "Circlip fitment", #  
Step 5 "Locking the circlip", # Step 6 "Drain plug fitment", # Step 7 "Apply Loctite", # Step 8  
"Cut-off valve fitment", # Step 9 "End cover tightening", # Step 10 "Drain plug tightening", #  
Step 11 "Cutoff valve tightening", # Step 12 "Detent Plunger", # Step 13 "Yellow Spring", #  
Step 14 "Pink Spring" # Step 15 ]
```

5.2 Detection Tracking Variables

Several variables track the current state of assembly:

cycle_check: List that stores completed steps (by index number)

cycle_check_f: Flag indicating cycle status (0=in progress, 10=complete)

sequence_break_f: Indicates if assembly sequence was broken (wrong order)

cls_ins (deque): Recent 30 detections, used to confirm consistent detection

tower_conf (deque): Recent 20 tower presence confirmations

spring_plungerf: Special flag for spring/plunger installation verification

5.3 Position Detection Boxes

Specific screen areas are monitored for hand positions during spring/plunger installation:

```
detent_box_pos = [41, 232, 142, 309] # [x1, y1, x2, y2] for detent plunger area  
yellowS_box_pos = [216, 110, 279, 168] # Yellow spring installation area pinkS_box_pos = [205,  
187, 271, 238] # Pink spring installation area
```


6. Core Functions Deep Dive

6.1 pyg() - Display Function

This function creates the visual overlay showing assembly progress. It displays:

- Current cycle status (In Progress, Completed, or Sequence Break)
- List of completed steps with green checkmarks
- Current step being performed (highlighted in yellow)
- Sequence break warnings (highlighted in red with blinking effect)
- Semi-transparent background for better readability

```
def pyg(frame, details_=[], time_=[]): # Creates visual overlay on video frame # Parameters: #
frame: The video frame to draw on # details_: List of completed step indices # time_:
Timestamps for each step (not currently used) # Color codes: yellow = [14, 201, 255] # Current
step (BGR format) green = [0, 255, 0] # Completed steps red = [0, 0, 255] # Sequence break
white = [255, 255, 255] # Text on red background # Draw semi-transparent background panel
overlay = frame.copy() cv2.rectangle(overlay, (600, 0), (960, height), [0, 0, 0], -1)
cv2.addWeighted(overlay, 0.6, frame, 0.4, 0, frame) # Draw each step with appropriate color
for i, step_name in enumerate(details_): if step_completed: cv2.circle(frame, (900, y), 10,
green, -1) # Green dot elif step_is_current: cv2.circle(frame, (900, y), 10, yellow, -1) #
Yellow dot # ... etc
```

6.2 frameInf() - Main Detection Function

This is the brain of the system. For each video frame, it:

- Step 1:** Runs YOLO detection on the frame to find objects
- Step 2:** Checks if the tower (main assembly) is present
- Step 3:** Verifies each assembly step in order
- Step 4:** Monitors drill positions for tightening operations
- Step 5:** Tracks hand positions for spring/plunger installation
- Step 6:** Detects sequence breaks (wrong order)

Step 7: Draws bounding boxes and status overlay

Step 8: Returns the annotated frame for display

7. Assembly Process Steps Explained

7.1 Step Detection Logic

Each step requires the object to be detected consistently (3+ times in recent frames) to avoid false positives from momentary detections. Here's how each step is validated:

End Cover (Step 0)

Detection: Detected when object class 0 appears 3+ times

Prerequisites: No prerequisites

End Cover Bolt (Step 1)

Detection: Detected when object class 1 appears 3+ times

Prerequisites: Requires Step 0 completed

Clamp (Step 2)

Detection: Detected when object class 3 appears 3+ times

Prerequisites: Can be done anytime after tower present

Interlock Pin (Step 3)

Detection: Detected when object class 5 appears 3+ times

Prerequisites: Starts a 30-second timer

Hammering (Step 4)

Detection: Detected when hammer (class 4) appears 2+ times

Prerequisites: Must immediately follow Step 3

Circlip (Step 5)

Detection: Detected when object class 6 appears 3+ times

Prerequisites: Requires 30 seconds after Step 3

Lock Circlip (Step 6)

Detection: Detected when hammer appears after circlip

Prerequisites: Must immediately follow Step 5

Drain Plug (Step 7)

Detection: Detected when object class 7 appears 3+ times

Prerequisites: Independent step

Apply Loctite (Step 8)

Detection: Detected when loctite (class 8) appears 3+ times

Prerequisites: Requires Step 7 completed

Cut-off Valve (Step 9)

Detection: Detected when object class 9 appears 3+ times

Prerequisites: Independent step

Tightening (Steps 10-12)

Detection: Detected by drill position tracking

Prerequisites: Monitors drill coordinates

7.2 Sequence Break Detection

The system enforces strict ordering for critical steps. A sequence break occurs when:

- Installing circlip (Step 5) before hammering interlock (Step 4)
- Performing any step that requires a prerequisite before that prerequisite is complete
- The `sequence_break_f` variable stores which step number the break occurred at
- When a break is detected, the affected step blinks red in the display

7.3 Special Cases: Tightening Detection

Tightening operations (Steps 10-12) use position-based detection. The system monitors drill tool center coordinates and checks if they fall within specific regions:

```
# End cover tightening (Step 10) if 480 < drill_y < 520 and 145 < drill_x < 180:
cls_ins.append(30) # Mark as end cover tightening # Cut-off valve tightening (Step 12) if 195
< drill_y < 255 and 795 < drill_x < 820: cls_ins.append(32) # Mark as cutoff valve tightening #
Drain plug tightening (Step 11) if 250 < drill_y < 285 and 615 < drill_x < 635:
cls_ins.append(31) # Mark as drain plug tightening
```

7.4 Spring/Plunger Installation (Steps 13-15)

These steps use hand position tracking. The system:

- Detects when hands enter specific installation zones
- Tracks if both hands are positioned correctly around the installation point
- Measures distance between hands to confirm proper positioning
- Sets `spring_plungerf` flag when installation is verified
- Clears the flag when hands move away or next part is installed

8. Web Server Implementation

8.1 Flask Routes

The system exposes several web endpoints for control and monitoring:

Route	Method	Function	Description
/	GET	index()	Returns "Ashok Leyland" message
/clear	GET	cls()	Clears cycle_check list, resets progress
/record	GET	record()	Toggles video recording on/off
/live	GET	index_t()	Returns HTML page to display video
/st	GET	getF()	Starts video capture thread
/video_feed	GET	video_feed()	Streams processed video frames

8.2 Video Streaming

The /video_feed endpoint implements MJPEG streaming:

```
def generate_frames(): # Generator function that yields frames continuously while True: #
Process current frame res = frameInf(capFrame.copy()) # Encode frame as JPEG _, buffer =
cv2.imencode('.jpg', res) # Yield as multipart HTTP response yield (b'--frame\r\n'
b'Content-Type: image/jpeg\r\n\r\n' + buffer.tobytes() + b'\r\n') @app.route('/video_feed')
def video_feed(): return Response(generate_frames(), mimetype='multipart/x-mixed-replace;
boundary=frame')
```

This creates a continuous stream of JPEG images that browsers can display as live video. Each frame is sent as part of a multipart HTTP response.

9. Video Processing Pipeline

9.1 Video Capture with PyAV

The system uses PyAV instead of OpenCV for better RTSP performance:

```
def getF(): # RTSP camera URL with credentials video_path =  
"rtsp://sourab:tvsm123!@192.168.0.101/stream1?tcp" # Open video container with hardware  
acceleration container = av.open(video_path, options={ "rtsp_transport": "tcp", # Use TCP for  
reliability "hwaccel": "cuda" # Use GPU acceleration }) # Continuous capture loop while True:  
try: for frame in container.decode(video=0): # Convert to numpy array (BGR format) frame =  
frame.to_ndarray(format="bgr24") # Crop to region of interest frame = frame[450:1500,  
550:2000] # Store in global variable capFrame = frame.copy() except Exception as e: #  
Reconnect on error time.sleep(5) container = av.open(video_path, options={...})
```

9.2 Frame Processing Flow

1. **Capture:** getF() continuously captures frames from RTSP stream 2. **Store:** Latest frame stored in global capFrame variable 3. **Request:** Web client requests video_feed endpoint 4. **Process:** generate_frames() calls frameIn() to process capFrame 5. **Detect:** YOLO models detect objects in frame 6. **Analyze:** Detection logic validates assembly sequence 7. **Annotate:** Visual overlays added to frame 8. **Encode:** Frame encoded as JPEG 9. **Stream:** JPEG sent to web client 10. **Repeat:** Process continues for next frame

10. Detection Logic Explained

10.1 YOLO Model Configuration

The system uses two YOLO models:

Pose Model (yolov8n-pose.pt)

Purpose: Detects human poses and hand positions

Usage: Used for spring/plunger installation verification

Tower Model (AL_tower_271225.pt)

Purpose: Detects assembly parts and tools

Usage: Custom trained for this specific application

10.2 Object Class Mappings

The tower model is trained to detect these object classes:

Class ID	Object Name	Assembly Step
0	end_cover	Step 0: End cover fitment
1	end_cover_bolt	Step 1: End cover bolt fitment
2	tower	Main assembly (presence check)
3	clamp	Step 2: Clamping
4	hammer	Steps 4, 6: Hammering operations
5	lock_pin_interlock	Step 3: Interlock pin fitment
6	circlip_interlock	Step 5: Circlip fitment
7	drain_plug	Step 7: Drain plug fitment
8	loctite	Step 8: Apply Loctite
9	cutoff_valve	Step 9: Cut-off valve fitment
14	drillm	Steps 10-12: Tightening operations
15	hand_gloves	Steps 13-15: Hand position tracking
30	end_cover_tightening	Step 10 (position-based)
31	drain_plug_tightening	Step 11 (position-based)
32	cutoff_valve_tightening	Step 12 (position-based)

10.3 Detection Confidence Mechanism

To avoid false positives from brief or accidental detections, the system uses a deque (double-ended queue) to track recent detections:

```
cls_ins = deque(maxlen=30) # Stores last 30 detections tower_conf = deque(maxlen=20) # Stores
last 20 tower detections # Detection is confirmed when object appears 3+ times in recent
history if cls_ins.count(object_class) >= 3: # Confirmed detection, proceed with step logic
cycle_check.append(step_number)
```

This means an object must be detected in at least 3 of the last 30 frames (10% consistency) before being considered a valid detection. This filters out momentary false positives while still responding quickly to actual part installations.

11. Troubleshooting and Best Practices

11.1 Common Issues

Video stream not connecting

- Check camera IP address and credentials
- Verify network connectivity
- Ensure RTSP port is not blocked by firewall
- Try changing rtsp_transport from 'tcp' to 'udp'

Detection not working

- Verify model files exist in models/ directory
- Check camera angle and lighting
- Ensure objects are clearly visible
- Review detection confidence threshold (currently 0.4)

Sequence breaks incorrectly triggered

- Adjust detection count threshold (currently 3)
- Increase deque size for more stable detection
- Review lighting conditions affecting detection
- Check for objects in frame that confuse the model

Web interface not loading

- Verify Flask server is running (should print 'Running on http://0.0.0.0:5000')
- Check firewall settings on server
- Access from browser: `http://[server-ip]:5000/live`
- Check server logs for errors

Recording not working

- Verify output directory exists: `C:/Users/admin/Videos/Recorded/`
- Check disk space availability
- Ensure codec 'H264' is installed
- Check file permissions on output directory

11.2 Performance Optimization

- **Use GPU acceleration:** Ensure CUDA is properly configured for both PyAV and YOLO
- **Adjust frame size:** Current crop is [450:1500, 550:2000] - smaller regions process faster
- **Lower YOLO image size:** Currently 640x640, can reduce to 416x416 for faster processing
- **Adjust confidence threshold:** Higher confidence (0.5-0.6) = fewer false positives but may miss some detections
- **Threading optimization:** System uses threading for video capture - ensure thread isn't blocking
- **Network optimization:** Use TCP transport for reliability, UDP for speed (if network is stable)

11.3 Best Practices

- **Camera Placement:** Ensure clear, unobstructed view of entire assembly area
- **Lighting:** Use consistent, bright lighting to improve detection accuracy
- **Model Training:** Regularly retrain models with new examples to improve accuracy
- **Monitoring:** Regularly review recorded videos to catch and fix detection issues
- **Maintenance:** Clear cycle_check periodically using /clear endpoint between assemblies
- **Testing:** Test sequence break detection regularly to ensure safety compliance
- **Backup:** Keep backups of trained models and configuration settings
- **Documentation:** Update detection coordinates if camera position changes

11.4 System Requirements

Minimum system specifications for running this monitoring system:

Component	Requirement	Recommended
-----------	-------------	-------------

CPU	Quad-core 2.5GHz+	Intel i7 or AMD Ryzen 7
RAM	8GB	16GB or more
GPU	NVIDIA with CUDA support	RTX 3060 or better
Storage	50GB free space	SSD with 100GB+ free
Network	100Mbps Ethernet	Gigabit Ethernet
OS	Ubuntu 20.04+	Ubuntu 22.04 LTS
Python	Python 3.8+	Python 3.10 or 3.11
CUDA	CUDA 11.0+	CUDA 12.0+

Quick Reference Guide

Essential URLs

When the system is running on IP address 192.168.1.100:

Live view: <http://192.168.1.100:5000/live>

Video stream: http://192.168.1.100:5000/video_feed

Clear progress: <http://192.168.1.100:5000/clear>

Toggle recording: <http://192.168.1.100:5000/record>

Start capture: <http://192.168.1.100:5000/st>

Startup Sequence

1. Ensure camera is powered and accessible on network 2. Activate Python environment with dependencies 3. Run: `python app.py` 4. Wait for "Running on <http://0.0.0.0:5000>" message 5. Open browser to [http://\[server-ip\]:5000/live](http://[server-ip]:5000/live) 6. System will automatically start processing video

Color Code Legend

Green: Step completed successfully **Yellow:** Current step in progress **Red:** Sequence break detected (wrong order) **White on Red:** Step causing sequence break (blinking)

Support Contact

For technical support or questions about this system, contact your IT department or system administrator. Keep model files, logs, and recorded videos for troubleshooting purposes.

Assembly Line Monitoring System - Computer Vision Guide
Document Version 1.0 - January 2026